Project Assignment 3

March 12, 2026

0.1 Introduction and Data Sourcing

For this project, I will be showing a map of severe thunderstorm warnings based on their respective Weather Forecasting Office (also called WFO for short). With this information, I am going to load in a shapefile of damage associated with severe weather, and see if the warnings and damage reports show any significance on a heatmap. I will also add in population data and analyze it by WFO to see if there is more of a correlation of damage with where there are more people. Essentially, are there more damage reports where there are more severe thunderstorm warnings, or where there is more infrastructure or people? CSV values of 2025 severe thunderstorm warnings come from the Iowa State Mesonet, and are found at this link: https://mesonet.agron.iastate.edu/vtec/search.php?mode=list&by=wfo&year=2025&phenomena=SV&significanc . The WFO shapefiles come from the National Weather Service, and can be found at this link: https://www.weather.gov/gis/NWSRegions . The damage points come from NOAA as well, and are in a csv file of all the damage reports from 2025: https://www.ncei.noaa.gov/pub/data/swdi/stormevents/csvfiles/ . The population data can be found at the US Census office, and at this link: https://www.census.gov/cgi-bin/geo/shapefiles/index.php .

0.2 Importing the Packages and the Data

[1]:
```
%matplotlib inline
import os
from datetime import datetime

os.environ["PROJ_LIB"] = r'/opt/conda/pkgs/proj4-5.2.0-he1b5a44_1006/share/proj/
  ↪'

import numpy as np
import mpl_toolkits

import pandas as pd
from shapely.geometry import Point

import mapclassify
import matplotlib.pyplot as plt
import matplotlib.patches as patches

import cartopy.crs as ccrs # import projection
```

```
import cartopy.feature as cf # import features

import json
import folium
import mplleaflet

import rasterio

from geopandas import GeoSeries, GeoDataFrame, read_file, gpd
from pandas import Series
import IPython
import fiona
import pprint
```

[2]:
```
tstorm = pd.read_csv('/home/jovyan/work/GGIS407/Project/tstorm.csv') #reads in␣
 ↪csv file of severe thunderstorm warnings
reports = pd.read_csv('/home/jovyan/work/GGIS407/Project/damagereports.csv')␣
 ↪#reads in the storm reports
wfo = gpd.read_file('/home/jovyan/work/GGIS407/Project/wfo.shp') #reads in␣
 ↪shapefile of all Weather Forecasting Offices in the USA
count = gpd.read_file('/home/jovyan/work/GGIS407/Project/tl_2024_us_county.
 ↪shp') #reads in all the counties to help plot the severe thunderstorm␣
 ↪warnings
pop = gpd.read_file('/home/jovyan/work/GGIS407/Project/popest.csv',␣
 ↪encoding='latin1') #population estimates by county, using POP_ESTIMATE_2023
```

0.3 Cleaning and Sorting the Data

A lot of the data came with non-necessary add ons, so below is me cleaning it up for the sake of the data processing. I only included the 2023 census estimates from the census data. I also gave spatial components to data that didn't have any. The severe thunderstorm counts were something I was already familiar with, so I added that in to count the severe thunderstorm warnings by WFO. The damage report data was for all weather events, so I filtered out any events that did not correlate with severe thunderstorm warnings, and made sure there was a money value (and not zero). I also counted the population by WFO below as well, as there was no defined dataset for that.

[3]:
```
#sorting the population csv file so it only has census data by county from 2023␣
 ↪estimates
pop = pop[pop['Attribute'] == 'POP_ESTIMATE_2023']

pop = pop[pop['Area_Name'].str.contains('County', na=False)]

pop = pop[['State', 'Area_Name', 'Value']]

pop.to_csv('counties_population_2023.csv', index=False)

print(pop.head())
```

```
    State        Area_Name   Value
126    AL  Autauga County   60342
191    AL  Baldwin County  253507
256    AL  Barbour County   24585
321    AL     Bibb County   21868
386    AL   Blount County   59816
```

[4]:
```
#setting the SRID so that the data has a spatial component to it
wfo = wfo.set_crs(epsg=4326)
count = count.set_crs(epsg=4269)
count = count.to_crs(epsg=4326)
```

[5]:
```
#limiting the area of the forecasting offices to just the continental United␣
 ↪States, for simplicity (and also lack of severe thunderstorm warnings in AK␣
 ↪and HI)
wfo_conus = wfo.cx[-125:-66, 24:50]
```

[6]:
```
#code originated from a data analysis project from ATMS 517, where i counted␣
 ↪flash flood warnings by county
tstorm_counts = tstorm.groupby('WFO').size().reset_index(name='warning_count')
wfo_merged = wfo.merge(tstorm_counts, on='WFO', how='left')
wfo_merged['warning_count'] = wfo_merged['warning_count'].fillna(0)
```

[11]:
```
#filtering the csv to only severe thunderstorm related damages
severe_types = ['Thunderstorm Wind', 'Thunderstorm Wind Damage', 'Thunderstorm␣
 ↪Wind Gust', 'Hail']
severe_sort = reports[reports['EVENT_TYPE'].isin(severe_types)]
damage = severe_sort[(severe_sort['DAMAGE_PROPERTY'] != '0') |
                     (severe_sort['DAMAGE_CROPS'] != '0')] #dollar value of␣
 ↪damage is defined
```

[12]:
```
pop['Value'] = pd.to_numeric(pop['Value'], errors='coerce') #convertiung the␣
 ↪population into a number value
pop['county'] = pop['Area_Name'].str.replace(' County', '', regex=False)␣
 ↪#county column formatting so the wfo can recognize
count_pop = count.merge(pop, left_on='NAME', right_on='county', how='left')␣
 ↪#merging the counties with the population
count_pop = count_pop.to_crs(wfo.crs) #converting to crs
count_pop['centroid'] = count_pop.geometry.centroid #taking the centroid␣
 ↪geometries of the county populations
count_pop = count_pop.set_geometry('centroid') #setting the geometries
count_wfo = gpd.sjoin(count_pop, wfo[['WFO','geometry']], op='within',␣
 ↪how='left') #spatially joining the population and the counties
wfo_pop = count_wfo.groupby('WFO')['Value'].sum().reset_index() #adding up all␣
 ↪the populations into the WFO
wfo_final = wfo.merge(wfo_pop, on='WFO', how='left') #merging the wfo_pop with␣
 ↪the wfo values
```

```
geojson = wfo_final.to_json() #converting to json
```

```
/tmp/ipykernel_309/2002263422.py:5: UserWarning: Geometry is in a geographic
CRS. Results from 'centroid' are likely incorrect. Use 'GeoSeries.to_crs()' to
re-project geometries to a projected CRS before this operation.

  count_pop['centroid'] = count_pop.geometry.centroid #taking the centroid
geometries of the county populations
```

0.4 Mapping the Data

Maps are created in folium, to make them as interactive as possible. The first map created is the damage reports by severe thunderstorm warning counts by WFO, and the second is the damage reports by WFO population. The maps are saved via html, to allow for ease of access and for the notebook is saved. If using this as a .ipynb file, feel free to comment out the save html function, and remove the # from the m. The maps will show up inside the notebook, but may be more difficult to use the notebook in the longer term.

[13]:
```
#creating the folium map
m = folium.Map(location=[39, -98], zoom_start=4)

#the choropleth heatmap is wfo_merged
folium.Choropleth(
    geo_data=wfo_merged,
    name='Severe Thunderstorm Warnings',
    data=wfo_merged,
    columns=['WFO', 'warning_count'],
    key_on='feature.properties.WFO',
    fill_color='YlOrRd',
    fill_opacity=0.7,
    line_opacity=0.3,
    legend_name='Severe Thunderstorm Warnings (2025)'
).add_to(m)

#creating the boundaries of the WFO offices
folium.GeoJson(
    wfo_merged,
    name="WFO Boundaries",
    style_function=lambda x: {
        'fillColor': 'transparent',
        'color': 'black',
        'weight': 0.5
    }
).add_to(m)

#hovering over brings the WFO severe thunderstorm warning counts
folium.GeoJson(
    wfo_merged,
```

```
    name="WFO Info",
    tooltip=folium.GeoJsonTooltip(
        fields=['WFO', 'warning_count'],
        aliases=['WFO:', 'Warnings:']
    )
).add_to(m)

#creates the layer for the damage reports
damage_layer = folium.FeatureGroup(name="Severe Thunderstorm Damage Reports")

#creating the damage reports
for _, row in damage.iterrows():
    if pd.notnull(row['BEGIN_LAT']) and pd.notnull(row['BEGIN_LON']):
        folium.CircleMarker(
            location=[row['BEGIN_LAT'], row['BEGIN_LON']],
            radius=3,
            color='blue',
            fill=True,
            fill_opacity=0.6,
            popup=f"""
            Event: {row['EVENT_TYPE']}<br>
            Property Damage: {row['DAMAGE_PROPERTY']}<br>
            """
        ).add_to(damage_layer)

#adding the layer to the folium map
damage_layer.add_to(m)

# Layer control
folium.LayerControl().add_to(m)

# Save map
m.save('severe_tstorm_wfo_heatmap.html')
```

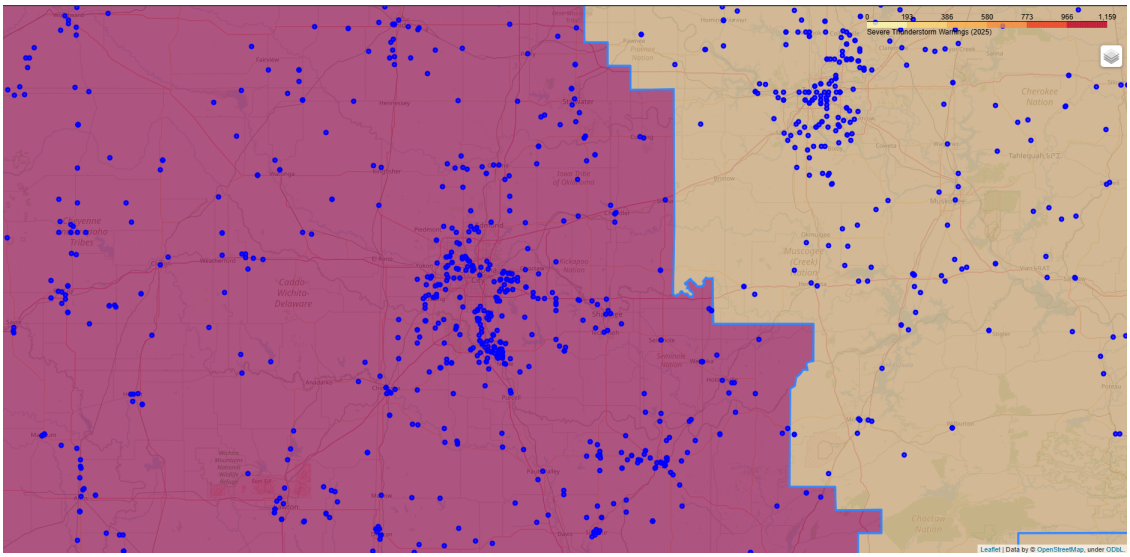
[14]:
```
m = folium.Map(location=[39, -98], zoom_start=4)

#creating the choropleth for the population counts by wfo
folium.Choropleth(
    geo_data=geojson,
    data=wfo_final,
    columns=['WFO','Value'],
    key_on='feature.properties.WFO',
    fill_color='YlOrRd',
    legend_name='Population by WFO'
).add_to(m)

#creates the layer for the damage reports
```

```
damage_layer = folium.FeatureGroup(name="Severe Thunderstorm Damage Reports")

#creating the damage reports
for _, row in damage.iterrows():
    if pd.notnull(row['BEGIN_LAT']) and pd.notnull(row['BEGIN_LON']):
        folium.CircleMarker(
            location=[row['BEGIN_LAT'], row['BEGIN_LON']],
            radius=3,
            color='blue',
            fill=True,
            fill_opacity=0.6,
            popup=f"""
            Event: {row['EVENT_TYPE']}<br>
            Property Damage: {row['DAMAGE_PROPERTY']}<br>
            """
        ).add_to(damage_layer)

#adding the layer to the folium map
damage_layer.add_to(m)

# Layer control
folium.LayerControl().add_to(m)

# Save map
m.save('damagepop.html')
#m
```

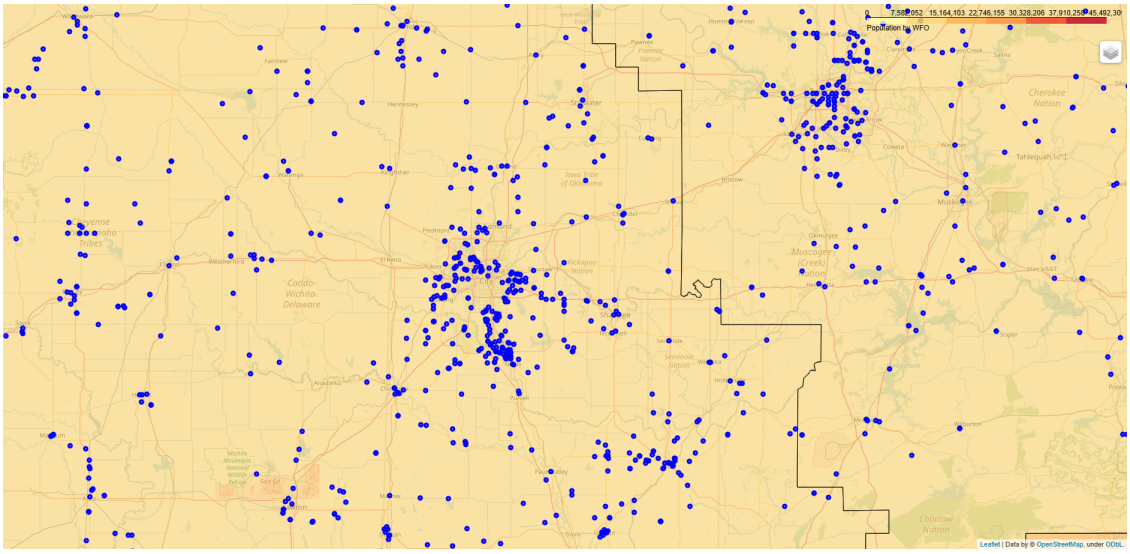
0.5 Conclusion

Since the interactive maps are too large to show on the code, these screenshots taken from two separate areas are shown instead. Oklahoma City is in the OUN forecasting region, which had 1,159 severe thunderstorm warnings in 2025. There is also a considerable amount of damage reports, but the population does not seem to be the major factor to the damage reports. In Florence, Alabama, there is less of a population impact, but still a considerable amount of severe thunderstorm warnings and damage within the region. Without any further assessment into climatology, urbanization, and population growth, this is the limits of the analysis. However, it is still a good visual to see the true extent of severe thunderstorm warnings and their damage.

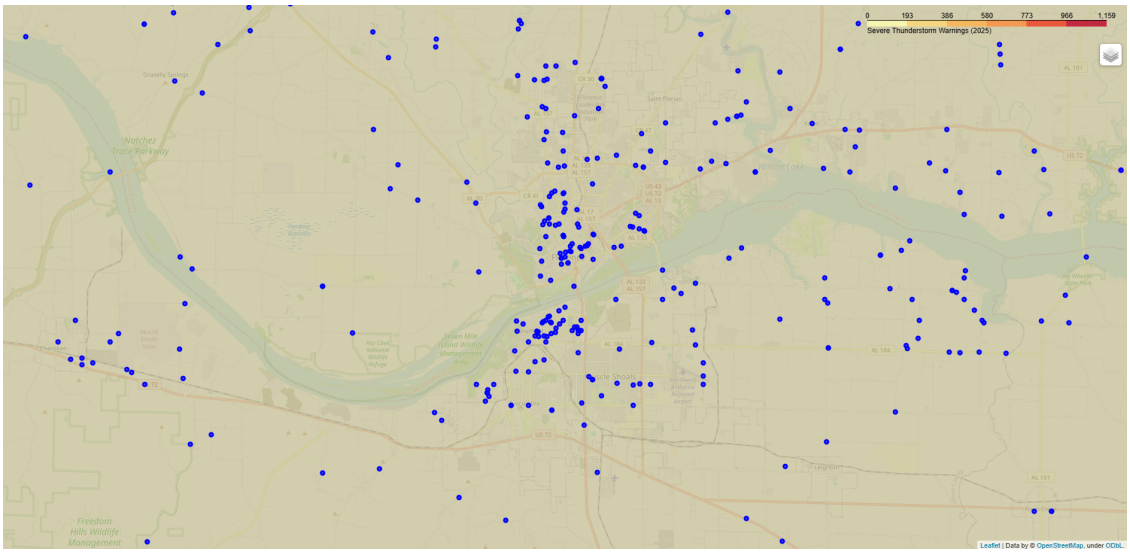
0.5.1 Damage Reports and Severe Thunderstorm Warning Counts by WFO near Oklahoma City, Oklahoma



0.5.2 Damage Reports and Population Counts by WFO near Oklahoma City, Oklahoma



0.5.3 Damage Reports and Severe Thunderstorm Warning Counts by WFO near Florence, Alabama



0.5.4 Damage Reports and Population Counts by WFO near Florence, Alabama